

FlappyBird PCG: Quality-Diversity Level Generation

Kasia

February 1, 2026

Agenda

- 1 Mechanika Gry
- 2 Boty
- 3 PCG Overview
- 4 MAP-Elites

Główny Fokus

Generatory poziomów (MAP-Elites, MCTS-QD, Novelty Search)

Parametry fizyczne:

- Grawitacja: 1.0 units/frame
- Siła skoku: -22 (accel. w górę)
- Prędkość terminalna: 40.0
- Hitbox: 5×3 (scale 0.8)

```
def update(self, dt: float)
    -> None:
    """Update bird physics
       """
    self.velocity += self.
        gravity
    self.velocity = min(
        self.velocity, self
        .terminal_velocity)
    self.y += self.velocity
        * dt

def jump(self) -> bool:
    """Execute jump if
       conditions allow"""
    if self.velocity >= 0:
        self.velocity =
            self.jump_force
    return True
```

AABB (Axis-Aligned Bounding Box)

Komponenty rury

- Body (główna część)
- Caps (zakończenia)
- Gap (przerwa do przelatywania)

Hitbox calculation:

$$\text{hitbox_width} = \text{width} \times 0.8$$

$$\text{hitbox_height} = \text{height} \times 0.8$$

Item collection: radius-based proximity check

Pipes:

- x (pozycja)
- gap_center (0-24)
- gap_size (6.0-11.0)
- spacing (35-52 units)

Items:

- Coins (regular, gold)
- Powerups (shield, small)
- Debuffs (large, speed)

Parametry poziomu:

- Length: **900 units**
- Pipes: 15-19
- Czas gry: ~30 sek

Behavior Features:

- Gap Tightness
- Spacing Density

Porównanie Strategii Botów

Bot	Strategia	Survival Rate
Aggressive	Minimize jumps, low altitude	60-70%
Reactive	React to current gap	50-60%
Coin Collector	A* pathfinding	95%+
Flat	Fixed height (test)	0% (varied)

Evaluation

Boty używane do oceny jakości poziomów:

- Playability: ~50% survival rate
- Balance: Bot performance variance
- Flat bot penalty: Wykrywa "flat" levels

Strategy: Prediction-based decision making

```
def decide_action(self):  
    """Prediction-based decision"""  
    bird = self.game.bird  
    next_pipe = self._find_next_pipe(pipes, bird.x)  
  
    gap_middle = (next_pipe.y_top + next_pipe.y_bottom  
                  ) / 2  
    risky_target = gap_middle +  
                    GAP_OFFSET_BELOW_MIDDLE  
  
    predicted_y = self._predict_position(  
        bird.y, bird.velocity, PREDICTION_STEPS=8  
    )  
  
    # Jump if predicted position would be too low  
    if predicted_y > risky_target + 1:  
        return JUMP
```

Strategy: 7-step lookahead with heuristic

```
def heuristic(self, state, pipes, items):  
    """A* heuristic for coin collector bot"""  
    # f(state) = g(state) + h(state)  
    h_score = 0  
  
    # Safety penalty  
    collision_penalty = self._check_collision_penalty(  
        state, pipes)  
    if collision_penalty > 0:  
        return collision_penalty  
  
    # Coin rewards (negative = good)  
    coin_reward = self._calculate_coin_reward(state,  
        items)  
    h_score += coin_reward  
  
    # Gap navigation penalty  
    h_score += self._calculate_navigation_penalty(  
        state, pipes, items)
```


Procedural Content Generation - Problem

Cel

Generuj zróżnicowane, **wysokiej jakości** poziomy

Challenge

Balance between **Quality** and **Diversity**

Rozwiązania: 3 algorytmy Quality-Diversity

- 1 MAP-Elites - Grid-based QD
- 2 MCTS-QD - Tree search + QD
- 3 Novelty Search - Pure diversity

Różne podejścia → różne trade-offs

Quality-Diversity

Maintain archive of **elite solutions** across behavior space

Archive: 6×6 grid (36 cells)

Behavior dimensions:

$$\text{Gap Tightness} = \frac{11 - \text{avg_gap}}{5} \in [0, 1]$$

$$\text{Spacing Density} = \frac{52 - \text{avg_spacing}}{17} \in [0, 1]$$

- Każda komórka: najlepszy poziom dla danego zachowania
- Mutacja: $\text{rate}=0.45$, $\sigma=0.25$
- Inicjalizacja: 250 random genomes

MAP-Elites: Algorytm

```
1: Initialize archive with 250 random genomes
2: for  $i = 1$  to 2000 do
3:   parent  $\leftarrow$  archive.get_random_elite()
4:   offspring  $\leftarrow$  parent.mutate(rate=0.45,  $\sigma=0.25$ )
5:   quality, behavior  $\leftarrow$  evaluate(offspring)
6:   cell  $\leftarrow$  get_cell_index(behavior)
7:   if archive[cell] is None OR quality > archive[cell].quality then
8:     archive[cell]  $\leftarrow$  offspring
9:   end if
10: end for
```

Kluczowe: Każda iteracja potencjalnie wypełnia jedną komórkę

$$Q = 0.35 \cdot P + 0.35 \cdot B + 0.30 \cdot R - P_{flat} - P_{vert} \quad (1)$$

Komponenty:

- P (Playability): ~50% survival, progress, no instant death
- B (Balance): Bot performance ratio (ideal: $1.0-1.2\times$)
- R (Progression): Smooth difficulty curve
- P_{flat} (Flat bot penalty): Wykrywa "flat" levels
- P_{vert} (Vertical penalty): Zbyt duża wariancja wysokości

Range

Quality $\in [-1.0, 1.0]$ (praktycznie: 0.3-0.8)

Statystyki:

- Coverage: **60-80%** (24-29/36 cells)
- Quality range: 0.3-0.8
- Max quality: **0.73**
- Iterations: 2000
- Time: ~5 min

Zalety

- Gwarantuje diversity (pełne pokrycie behavior space)
- Wysokie max quality
- Prosty i niezawodny

Wady

Powolna konwergencja (2000 iteracji)

Monte Carlo Tree Search + Quality-Diversity Archive

State-Action Space

- **State:** partial level (pipes built so far)
- **Action:** add next pipe (gap_size, gap_center, spacing, items)
- **Actions per state:** $6 \times 5 \times 5 \times 4 = 600$

Parametry:

- Simulations per level: 200-300
- Exploration constant: $C = 1.414 (\sqrt{2})$
- Target pipes: 17 (range: 15-19)

Tree search → lepsze próbki → sample efficiency

$$\text{UCT}(\text{node}) = \underbrace{\frac{Q}{N}}_{\text{exploit}} + C \underbrace{\sqrt{\frac{\ln N_{\text{parent}}}{N}}}_{\text{explore}} \quad (2)$$

where:

- Q = total reward
- N = visit count
- $C = 1.414$ (exploration constant)

Balans

- **Exploitation:** Wybierz akcje z wysokim average reward
- **Exploration:** Eksploruj rzadziej odwiedzane akcje

4 fazy (dla każdej symulacji):

- 1 **SELECT**: Traverse tree via highest UCT
- 2 **EXPAND**: Add new child node
- 3 **SIMULATE**: Random rollout to completion
- 4 **BACKPROPAGATE**: Update rewards up tree

```
def uct_value(self, exploration: float = 1.414) -> float:
    """UCT formula for node selection"""
    if self.visits == 0:
        return float('inf')

    exploitation = self.total_reward / self.visits
    exploration_term = exploration * math.sqrt(
        math.log(self.parent.visits) / self.visits
    )

    return exploitation + exploration_term
```


MCTS-QD: Wyniki vs MAP-Elites

Metryka	MAP-Elites	MCTS-QD
Coverage	60-80%	30-50%
Max Quality	0.73	0.62
Iterations	2000	30-100
Time	5 min	10-15 min
Advantage	Diversity	Sample efficiency

Trade-off

MCTS-QD: mniej iteracji, ale każda iteracja droższa (tree search)

Kluczowa Różnica

Fitness = Novelty (not Quality!)

$$\text{novelty}(x) = \frac{1}{k} \sum_{i=1}^k \text{dist}(x, \text{neighbor}_i) \quad (3)$$

k-NN distance in behavior space (k=15)

Archive:

- **Struktura:** List (not grid), max 1000 individuals
- **Selection:** Najmniej nowatorski zastępowany przez bardziej nowatorski
- **Quality:** Zapisywana, ale nie optymalizowana

Novelty Search: Algorytm

```
1: Initialize: 200 random genomes
2: for  $i = 1$  to 1000 do
3:   parent  $\leftarrow$  archive.get_random()
4:   offspring  $\leftarrow$  parent.mutate()
5:   behavior  $\leftarrow$  evaluate(offspring)
6:   novelty  $\leftarrow$  k_NN_distance(behavior, k=15)
7:   if archive.size < 1000 OR novelty > min_novelty then
8:     archive.add(offspring)
9:   end if
10: end for
```

Kluczowe: Brak pressure na quality $\rightarrow$ pure exploration

Novelty Search: k-NN Implementation

```
def compute_novelty(self, behavior: Tuple[float, ...],
                    k: int = 15) -> float:
    """Compute novelty as average distance to k-
       nearest neighbors"""
    if len(self.individuals) < k:
        return float('inf')

    distances = [
        euclidean_distance(behavior, ind.behavior)
        for ind in self.individuals
    ]

    k_nearest = sorted(distances)[:k]
    return sum(k_nearest) / k

def euclidean_distance(b1, b2):
    return np.sqrt(sum((a - b) ** 2 for a, b in zip(b1,
                                                       b2))))
```

Statystyki:

- Archive size: 500-1000 individuals
- Behavior spread: **wysoka diversity**
- Quality: 0.2-0.8 (wide variance)
- Max quality: 0.79 (przypadkowy spike)
- Iterations: 1000

Zalety

- **Najwyższa diversity** - odkrywa nietypowe struktury
- Brak local optima w quality space
- Może znaleźć surprising solutions

Wady

Brak gwarancji quality - niektóre poziomy niskiej jakości

Algorytm	Archive	Selection	Advantage	Disadvantage
MAP-Elites	Grid 6×6	Quality	Diversity guarantee	Slow convergence
MCTS-QD	Grid 6×6	UCT + Quality	Sample efficient	Complex
Novelty	List ~ 1000	Novelty (k-NN)	Exploration	No quality guarantee

Use cases:

- **MAP-Elites:** Gdy potrzebujesz balanced diversity + quality
- **MCTS-QD:** Gdy masz ograniczone evaluations budget
- **Novelty Search:** Gdy eksploracja > quality

Quality vs Diversity Trade-off

Metryka	MAP-Elites	MCTS-QD	Novelty
Coverage	60-80%	30-50%	Highest
Max Quality	0.73	0.62	0.79*
Avg Quality	0.40-0.50	0.50	0.35
Diversity	High	Medium	Highest
Iterations	2000	30-100	1000

*Novelty max quality to przypadkowy spike, nie konsystentna optymalizacja

Obserwacja

Nie ma "najlepszego" algorytmu - zależy od celu

MAP-Elites:

- 60-80% filled cells
- Balanced coverage across behavior space
- Clear quality gradient

MCTS-QD:

- 30-50% filled cells
- Concentrated in high-reward regions
- Higher avg quality per cell

Novelty Search:

- Continuous scatter (not grid-based)
- Highest behavioral diversity
- Wide quality variance

Quality Distributions

MAP-Elites:

- Median: ~ 0.4
- Max: **0.73**
- Range: 0.3-0.7
- Consistent quality

MCTS-QD:

- Median: ~ 0.5
- Max: 0.62
- Range: 0.3-0.6
- **Highest avg quality**

Novelty Search:

- Median: ~ 0.35
- Max: 0.79
- Range: 0.2-0.8
- Wide variance

MAP-Elites

Najlepszy dla diversity + quality balance

- Gwarantuje pokrycie behavior space
- Wysokie max quality (0.73)
- Prosty i niezawodny

MCTS-QD

Najlepszy dla sample efficiency

- Mniej iteracji (~100 vs 2000)
- Wyższy avg quality
- Dobry gdy evaluations są drogie

Novelty Search

Najlepszy dla exploration

- Najwyższa behavioral diversity

Możliwe rozszerzenia:

① Hybrid algorithms

- MAP-Elites + MCTS initialization
- Novelty Search + quality threshold

② Higher-dimensional behavior spaces

- Dodatkowe wymiary: vertical variance, item density
- 3D/4D grid archives

③ Player preference learning

- Personalized quality metrics
- Human-in-the-loop evaluation

④ Inne domeny

- Platformers, puzzle games
- Multi-objective optimization

Główne Wnioski

- Quality-Diversity algorithms generują **zróżnicowane, wysokiej jakości** poziomy
- Różne algorytmy → różne trade-offs
- MAP-Elites: best overall balance
- MCTS-QD: best sample efficiency
- Novelty Search: best exploration

Dziękuję za uwagę!

Kod: github.com/user/FlappyBird